



Sum of Path Numbers (medium)

We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
- Code
 - Time complexity
 - Space complexity

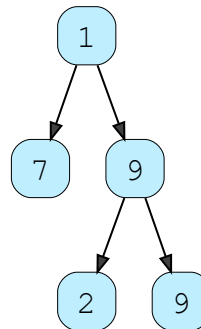
Problem Statement

Given a binary tree where each node can only have a digit (0-9) value, each root-to-leaf path will represent a number. Find the total sum of all the numbers represented by all paths.

Example 1:

Output: 408

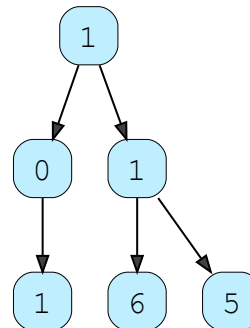
Explanation: The sum of all path numbers: $17 + 192 + 199$



Example 2:

Output: 332

Explanation: The sum of all path numbers: $101 + 116 + 115$



Tt

Try it yourself



Try solving this question here:

Java

Python3

JS

C++

```

1  import java.util.*;
2
3  class TreeNode {
4      int val;
5      TreeNode left;
6      TreeNode right;
7
8      TreeNode(int x) {
9          val = x;
10     }
11 };
12
13 class SumOfPathNumbers {
14     public static int findSumOfPathNumbers(TreeNode root) {
15         // TODO: Write your code here
16         return -1;
17     }
18
19     public static void main(String[] args) {
20         TreeNode root = new TreeNode(1);
21         root.left = new TreeNode(0);
22         root.right = new TreeNode(1);
23         root.left.left = new TreeNode(1);
24         root.right.left = new TreeNode(6);
25         root.right.right = new TreeNode(5);
26         System.out.println("Total Sum of Path Numbers: " + SumOfPathNumbers.findSumOfPathNumbers(root));
27     }
28 }
29

```

Run

Save

Reset

Solution

This problem follows the [Binary Tree Path Sum](#) pattern. We can follow the same **DFS** approach. The additional thing we need to do is to keep track of the number representing the current path.

How do we calculate the path number for a node? Taking the first example mentioned above, say we are at node '7'. As we know, the path number for this node is '17', which was calculated by: $1 * 10 + 7 \Rightarrow 17$. We will follow the same approach to calculate the path number of each node.

Code

Here is what our algorithm will look like:

Java

Python3

C++

JS

```

1  class TreeNode {
2      int val;
3      TreeNode left;
4      TreeNode right;
5
6      TreeNode(int x) {
7          val = x;
8      }
9  }

```

```

8   }
9   };
10
11  class SumOfPathNumbers {
12      public static int findSumOfPathNumbers(TreeNode root) {
13          return findRootToLeafPathNumbers(root, 0);
14      }
15
16      private static int findRootToLeafPathNumbers(TreeNode currentNode, int pathSum) {
17          if (currentNode == null)
18              return 0;
19
20          // calculate the path number of the current node
21          pathSum = 10 * pathSum + currentNode.val;
22
23          // if the current node is a leaf, return the current path sum.
24          if (currentNode.left == null && currentNode.right == null) {
25              return pathSum;
26          }
27
28          // traverse the left and the right sub-tree
29          return findRootToLeafPathNumbers(currentNode.left, pathSum) +
30                 findRootToLeafPathNumbers(currentNode.right, pathSum);
31      }
32
33      public static void main(String[] args) {
34          TreeNode root = new TreeNode(1);
35          root.left = new TreeNode(0);
36          root.right = new TreeNode(1);
37          root.left.left = new TreeNode(1);
38          root.right.left = new TreeNode(6);
39          root.right.right = new TreeNode(5);
40          System.out.println("Total Sum of Path Numbers: " + SumOfPathNumbers.findSumOfPathNumbers(root));
41      }
42  }

```

Run

Save

Reset



Time complexity

The time complexity of the above algorithm is $O(N)$, where 'N' is the total number of nodes in the tree. This is due to the fact that we traverse each node once.

Space complexity

The space complexity of the above algorithm will be $O(N)$ in the worst case. This space will be used to store the recursion stack. The worst case will happen when the given tree is a linked list (i.e., every node has only one child).

[< Back](#)
☒ Mark As Completed

[Next >](#)

All Paths for a Sum (medium)

Path With Given Sequence (medium)